

SENIOR AI ENGINEER INTERVIEW Q&A; PREPARATION

System Architecture, Decision-Making, and Performance Optimization

Project: Agentic RAG Multi-Format Chatbot Workspace

Target Role: Senior AI Engineer

Focus: Backend, Retrieval, Agents & Optimizations

Date: August 2026

1. System Latency & Graph Optimizations

Q1: Your initial local LLM RAG backend took minutes to return the first token for complex queries. How did you diagnose the bottleneck, and what architectural changes resolved it?

Answer: The bottleneck was diagnosed by profiling the LangGraph runtime, which revealed that background agent nodes (intent classification, query refinement, and reasoning) were executing synchronous, non-streamed LLM calls (`_llm.ainvoke`) sequentially *before* the streaming loop started. On local CPU/hardware, this introduced a 5-10 minute blocker.

Architectural Fixes:

- Zero-Delay Routing:** Replaced the LLM intent classifier in `query_analyzer.py` with a rule-based regex classifier. It categorizes queries (factual vs. tabular vs. visual) in 0 ms with high accuracy.
- Dynamic Graph Fast-Pathing:** Modified the Graph agents to immediately prioritize and structure context chunks in 0 ms instead of performing background LLM self-reflection steps.
- Single LLM Generation Loop:** Bypassed multi-step generation cycles so that retrieved context is assembled into a single prompt and streamed directly through `answer_stream` in `answer.py`. This reduced the time-to-first-token (TTFT) from minutes to **under 2 seconds**.

Q2: How did you implement intent classification without utilizing an LLM, and why is this a senior engineering choice?

Answer: I built a deterministic regex-based classifier that scans queries for structural and domain-specific keywords (e.g., matching `'page \d+'`, `'table'`, `'chart'`, `'summary'`, `'explain'`).

This is a senior-level engineering choice because it prioritizes **efficiency, latency, and cost-containment** over 'AI-for-everything' hype. In production RAG pipelines, calling an LLM for basic string classification adds unnecessary network overhead, increases token costs, and introduces non-deterministic routing bugs. Rule-based routers are faster, 100% predictable, and run locally at zero cost.

2. Advanced Retrieval Strategies & Fusion

Q3: Explain your hybrid search strategy. Why combine vector embeddings with lexical BM25, and how is the scoring fused?

Answer: Vector search (dense retrieval) excels at mapping conceptual and semantic meanings but often struggles with exact keyword matching (e.g., identifying part numbers, email addresses, or specific function names). BM25 (sparse retrieval) is highly precise for exact keywords but fails to capture synonyms or semantic intent.

I implemented a **hybrid search engine** in `retrieval.py`. The `retrieve` node queries the ChromaDB vector store and retrieves the top 15 results, then indexes the text using BM25. The final ranking is determined by normalized score fusion:
$$\text{Score} = (0.6 * \text{VectorSimilarity}) + (0.4 * \text{NormalizedBM25Score})$$

Both scores are explicitly cast to native floats to prevent serialization crashes, and the list is sorted in descending order to deliver highly precise contextual chunks.

Q4: How did you solve RAG contextual failure when users asked global document summary queries ('explain this PDF')?

Answer: Standard RAG fails global summaries because vector search retrieves a narrow set of chunks (e.g. 5-6 chunks), missing the broader document structure. To solve this, I built **Structural Document Retrieval**:

If a global query is detected (matching keywords like 'explain', 'summarize', 'overview' applied to the document), the retriever queries the vector store's metadata directly to extract:

1. The first 3 pages of the document (typically Title, Abstract, Introduction, and Table of Contents).
2. The last 2 pages of the document (Conclusions and Summary).
3. Combined with standard semantic search chunks.

These chunks are merged, duplicates are removed, and the full document blueprint is passed to the LLM. This provides a macro-level document understanding, matching the quality of commercial APIs like ChatGPT.

Q5: How does your system retrieve target-specific content when a query refers to a particular page ('explain the chart on page 75')?

Answer: I implemented **Targeted Page Mapping** in the retrieval layer. The retriever parses the query using regex to check for page indicators (e.g., 'page 12' or 'p. 75'). If a page number is extracted, the retriever performs a metadata query directly on ChromaDB: `where={'page_num': target_page}`. This guarantees that all text, tables, and image logs extracted from that exact page are loaded into context, bypassing standard embedding distance limitations.

3. Multimodal Ingestion & Ingestion Pipelines

Q6: How did you design a document parsing pipeline that handles scanned text, tables, and textless images, and how does the vision agent index them?

Answer: I built a streaming document parser in `parser.py` using PyMuPDF:

1. **Native Text & Tables:** If selectable text is present, it is extracted. Table structures are extracted via PyMuPDF's Native Table Finder, converted to Pandas DataFrames, and stored as Markdown tables for structured layout preservation.
2. **Scanned Fallback OCR:** If a page contains no machine-readable text, it is rendered to an image at 200 DPI and run through Pytesseract OCR.
3. **Multimodal Vision Agent:** If Tesseract returns empty text (indicating a diagram, chart, or photo), we trigger **Groq's llama-3.2-11b-vision-preview** model. The raw image bytes are sent to Groq, which returns a detailed visual description (e.g. 'A block diagram showing a Kali Linux instance targeting a Suricata firewall...'). This description is appended to the chunk text and indexed in ChromaDB so it can be queried normally.

Q7: How did you implement automatic Tesseract OCR discovery on Windows?

Answer: On Windows, Tesseract is a native binary that is not added to the user's PATH by default. To make the project run out-of-the-box, I wrote an auto-discovery block in `parser.py`. If tesseract is not in the system PATH, the script searches standard Windows installation directories (e.g., `C:\Program Files\Tesseract-OCR\tesseract.exe` and User AppData folders). If found, it dynamically registers it by setting `pytesseract.pytesseract.tesseract_cmd = path`. This eliminates manual configuration errors for Windows users.

4. Software Engineering & System Reliability

Q8: How did you solve the bug where LLM responses stopped by the user (Stop generation button) disappeared upon refresh or signing in again?

Answer: When a user clicks the Stop button, the frontend calls `abort()` on the active `AbortSignal`, closing the HTTP connection. In Python, this disconnect raises a `GeneratorExit` exception inside the FastAPI SSE generator (`event_stream`). Originally, the database write was placed after the streaming loop completed; since the generator exited early, the write was never reached.

To solve this, I refactored the database persistence in `backend/chat/router.py` to execute inside a `finally` block. Any partial response tokens that have been generated and accumulated up to that point are successfully saved to MongoDB before the connection thread terminates, guaranteeing session durability.

Q9: What is the benefit of using local embeddings (all-MiniLM-L6-v2) for ChromaDB and semantic caching while utilizing Groq APIs for the LLM?

Answer: This hybrid architecture balances **cost, latency, and throughput**:

1. **Zero Cost & Zero Network Latency:** Generating vector embeddings locally on CPU using all-MiniLM-L6-v2 is extremely fast (~10-20 ms per query) and 100% free, saving API costs on high-frequency semantic cache lookups.
2. **High-Speed LLM Inference:** Swapping local Ollama with Groq's llama-3.3-70b-versatile shifts the heavy text generation to Groq's LPU infrastructure, providing token generation speeds of over 100 tokens/sec. This delivers a premium user experience without requiring expensive local GPUs.

Q10: Why did you experience msgpack serialization errors (numpy.float64) when building the LangGraph checkpointer, and how did you debug it?

Answer: LangGraph's MemorySaver checkpointer serializes graph states using msgpack. Because my BM25 re-ranking logic used NumPy arrays, calculations like `combined_score = 0.6 * vector_score + 0.4 * bm25_normalized` returned `numpy.float64` objects instead of native Python floats. msgpack cannot serialize NumPy data types and threw a traceback crash.

I resolved this by tracing the data types of all scores in the retrieval collection and adding explicit float casting (e.g. `float(combined_score)`) before appending them to the state dictionaries.